

Week 1 — Lab Setup and Telemetry Pipeline

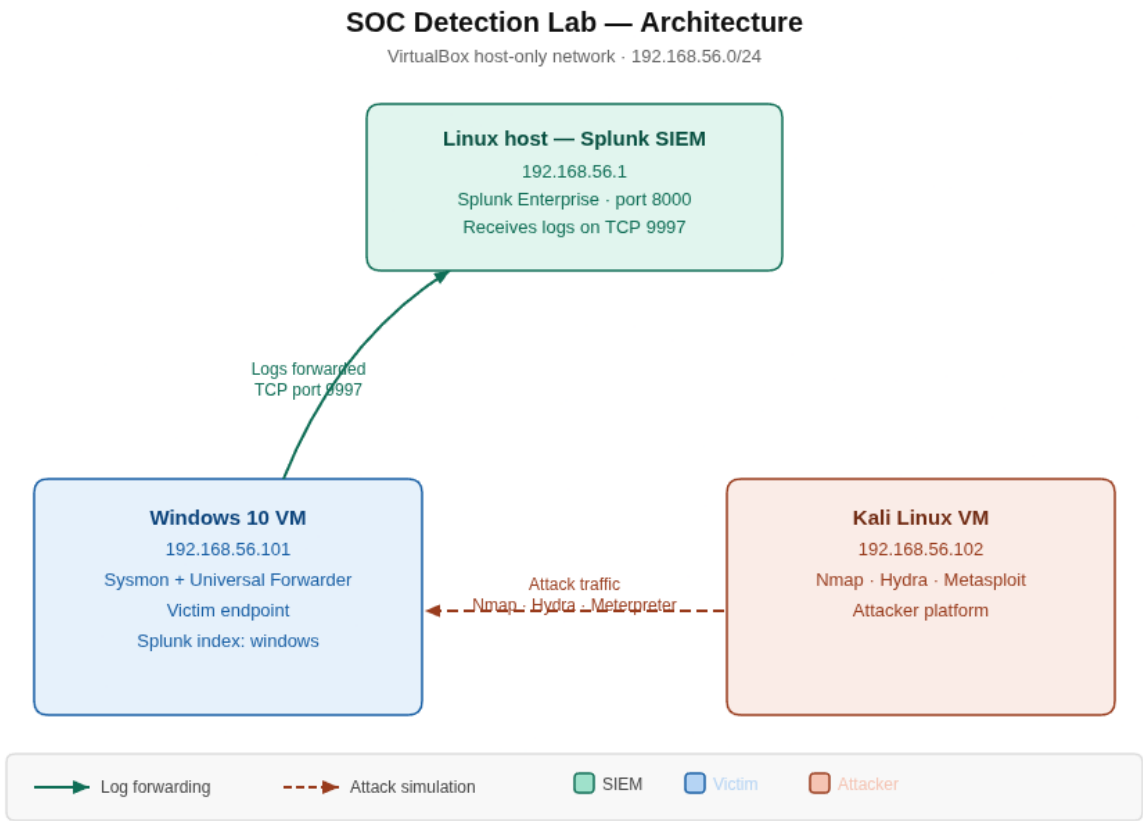
Objective

The objective of Week 1 was to build a working security telemetry pipeline that collects endpoint logs from a Windows 10 virtual machine and forwards them to a centralized Splunk SIEM for analysis. This establishes the foundation for all attack simulation and detection engineering work in subsequent weeks.

Lab Architecture

The lab consists of three systems communicating over a VirtualBox Host-Only network on the 192.168.56.0/24 subnet.

System	Role	IP Address
Linux Host	Splunk SIEM	192.168.56.1
Windows 10 VM	Monitored Endpoint	192.168.56.101
Kali Linux VM	Attacker	192.168.56.102



The Windows endpoint generates security telemetry which is forwarded to the Splunk server for indexing and analysis. The Kali VM is used in later weeks for attack simulation.

Log pipeline flow:

Windows 10 VM generates logs → Splunk Universal Forwarder collects logs → Logs forwarded over TCP port 9997 → Splunk Enterprise indexes events → Events searchable under index=windows

Issues Encountered and Resolutions

Issue 1 — VMs Could Not Communicate (NAT Networking)

Problem: Both VMs were initially configured with NAT networking. VMs on NAT are isolated behind their own network address translation layer and cannot communicate directly with each other or with the host machine.

Symptom: Both VMs displayed the same IP address 10.0.2.15, which is the default VirtualBox NAT gateway address. Ping between machines failed entirely.

Resolution: A Host-Only network was created in VirtualBox and a second network adapter was added to both VMs.

Steps taken:

1. VirtualBox → File → Tools → Network Manager → Create vboxnet0
2. Each VM → Settings → Network → Adapter 2 → Host-Only Adapter → vboxnet0
3. NAT adapter was retained on Adapter 1 to maintain internet access

After this change both VMs received 192.168.56.x addresses and could communicate with each other and the Linux host.

Lesson Learned: VirtualBox NAT is suitable for internet access but cannot be used for inter-VM communication. Host-Only networking is required for all internal lab traffic.

Issue 2 — Windows Firewall Blocking Ping from Kali

Problem: Ping from Kali to Windows failed even after Host-Only networking was configured. Ping from Windows to Kali succeeded, indicating Windows Firewall was blocking inbound ICMP requests.

Resolution:

```
netsh advfirewall firewall add rule name="Allow ICMPv4" protocol=icmpv4:8,any dir=in action=allow
```

Issue 3 — Splunk Dropping Events Due to Missing Index

Problem: The following error appeared in Splunk internal logs after initial forwarder configuration:

```
Received event for unconfigured index=windows with
source="WinEventLog:Windows PowerShell"
host="DESKTOP-6C9QB2R"
Dropping them as lastChanceIndex setting is not configured.
```

Root Cause: The Windows forwarder was configured to send logs to index=windows but that index had not been created in Splunk. All events were being dropped at the receiver.

Resolution: The index was created in Splunk Web:

Settings → Indexes → New Index → Name: windows → Save

Issue 4 — Forwarder Pointing to Stale IP Address

Problem: After reconfiguring the VM networking, the Splunk Universal Forwarder was still pointing to the old IP address 10.0.0.200 in outputs.conf. No data was reaching Splunk.

Resolution: The outputs.conf file on the Windows VM was updated to use the correct Host-Only IP:

```
[tcpout]
defaultGroup = primary_indexers

[tcpout:primary_indexers]
server = 192.168.56.1:9997

[tcpout-server://192.168.56.1:9997]
```

The forwarder was restarted after the change:

```
cd "C:\Program Files\SplunkUniversalForwarder\bin"
splunk restart
```

Step by Step Setup

Step 1 — Splunk Enterprise on Linux Host

Splunk Enterprise was installed on the Linux host and configured to receive forwarded logs on TCP port 9997.

Verify Splunk is running:

```
sudo /opt/splunk/bin/splunk status
```

Enable the receiving port in Splunk Web:

Settings → Forwarding and Receiving → Configure Receiving → New → 9997 → Save

Open the Linux firewall:

```
sudo ufw allow 9997/tcp
sudo ufw allow 8000/tcp
sudo ufw reload
```

Verify Splunk is listening:

```
sudo ss -lntp | grep 9997
```

Step 2 — Splunk Universal Forwarder on Windows

The Splunk Universal Forwarder was installed on the Windows 10 VM and configured to forward logs to the Splunk server on the Linux host.

outputs.conf — defines where logs are sent: check the file in /Configuration

Verify connectivity from Windows to Splunk:

```
Test-NetConnection 192.168.56.1 -Port 9997
```

Expected output:

```
TcpTestSucceeded : True
```

Restart forwarder after configuration:

```
cd "C:\Program Files\SplunkUniversalForwarder\bin"
splunk restart
```

Step 3 — Sysmon Installation on Windows

Sysmon was installed on the Windows VM to provide detailed endpoint telemetry beyond what native Windows logging offers. Sysmon captures process creation with full command lines, network connections per process, file creation events, and registry modifications.

Installation command:

```
Sysmon64.exe -accepteula -i sysmonconfig.xml
```

Verify the service is running:

```
Get-Service Sysmon64
```

Expected output:

Status	Name	DisplayName
Running	Sysmon64	Sysmon64

Step 4 — PowerShell Logging via Group Policy

PowerShell script block logging was enabled through Group Policy to capture PowerShell execution activity.

Path: Computer Configuration → Administrative Templates → Windows Components → Windows PowerShell

Settings enabled:

- Turn on Script Block Logging → Enabled
- Turn on Module Logging → Enabled

Step 5 — Windows Index Created in Splunk

A dedicated index was created in Splunk for all Windows telemetry:

Settings → Indexes → New Index → Name: windows → Save

Verification Queries

The following Splunk searches were run to confirm the telemetry pipeline was working correctly.

Check all active log sources:

```
index=windows  
| stats count by sourcetype
```

Result confirmed 4 active sourcetypes: WinEventLog:Application, WinEventLog:Security, WinEventLog:System, and WinEventLog:Windows PowerShell, with a total of 2,807 events.

Check authentication events:

```
index=windows source="WinEventLog:Security" EventCode=4624 OR EventCode=4625  
| table _time host Account_Name EventCode  
| sort _time
```

Check Sysmon process creation:

```
index=windows EventCode=1
| table _time Image CommandLine
| sort _time
```

Check Sysmon network connections:

```
index=windows EventCode=3
| table _time Image DestinationIp DestinationPort
| sort _time
```

Week 1 Outcome

By the end of Week 1 the following was confirmed:

- Splunk Enterprise running on the Linux host and accessible at <http://192.168.56.1:8000>
- Receiving port 9997 enabled and confirmed listening
- Splunk Universal Forwarder installed and running on the Windows VM
- Sysmon installed and generating detailed endpoint telemetry
- Windows Security, Sysmon, System, Application, and PowerShell logs all forwarding
- All events searchable in Splunk under index=windows
- Host-Only networking confirmed with bidirectional connectivity between all three machines

The telemetry pipeline was fully operational.